

Plataforma Fintech Descentralizada para la Gestión Financiera Grupal

Julián Duarte
Sebastián Ramos
Alejandra Rivera

Universidad Externado de Colombia

DeFi 1

28 de mayo de 2026

Introducción: Finanzas para humanos

Seguramente te ha pasado: estás en una cena increíble con amigos, la comida estuvo perfecta y la compañía inmejorable. Pero llega el momento de la cuenta y el ambiente se tensa. Empiezan las calculadoras de los celulares, los “yo solo pedí una entrada”, los “paga tú y mañana te envío” y la inevitable lluvia de comprobantes de transferencia por WhatsApp que nadie termina de revisar.

Este proyecto no nace solo de la tecnología, sino de la necesidad de devolverle la armonía a estos momentos. Queremos usar las herramientas más avanzadas de la actualidad como el **Fintech** y las **Finanzas Descentralizadas (DeFi)** para algo muy sencillo: que el dinero entre amigos deje de ser un dolor de cabeza. A través de la **Blockchain** y los **Smart Contracts**, buscamos que la confianza no dependa de una persona, sino de un código transparente que trabaje por nosotros.

La pesadilla de “la vaca”: Una historia conocida

Imagina que Alejandra, Sebastián y Julián deciden ir de viaje el próximo fin de semana. Como buenos amigos, acuerdan hacer una caja común para el transporte y el alojamiento. Alejandra, que siempre es la más organizada, se ofrece a recibir el dinero.

A partir de ahí, comienza la serie de eventos desafortunados:

- Julián envió la transferencia, pero el banco se demoró en reflejarla. Alejandra no sabe si creerle o no.
- Sebastián se olvidó de su parte y Alejandra tiene que cobrarle tres veces, lo cual es incómodo para ambos.
- Al final del viaje, sobra un poco de dinero, pero nadie recuerda exactamente cuánto puso cada uno al principio, por lo que la repartición se vuelve un caos.

Esta situación es el pan de cada día en grupos de amigos, familias y pequeños emprendimientos. El problema real no es la falta de dinero, sino la **falta de una estructura transparente**. Actualmente, dependemos de la memoria de alguien, de hilos de chat interminables y de una gestión manual que es lenta y estresante.

¿Y si hubiera una forma en que el dinero se administrara solo? ¿Y si mientras ahorramos para ese viaje, la plata pudiera ir creciendo sola mediante una inversión segura? De esa pregunta nace nuestra propuesta: una plataforma donde el dinero se mueve con la misma facilidad y confianza con la que fluye la conversación en esa cena con amigos.

Nuestros Objetivos: Hacia una gestión inteligente

Con este proyecto, no solo queremos construir una aplicación, sino establecer un nuevo estándar de confianza en cómo grupos de personas manejan su capital.

Objetivo General

Desarrollar un prototipo de plataforma fintech descentralizada que automatice la recolección, administración e inversión de fondos grupales, garantizando transparencia total mediante contratos inteligentes.

Objetivos Específicos

- **Automatizar la gestión de gastos:** Diseñar un sistema que calcule y recolecte de forma exacta los aportes individuales para gastos compartidos, eliminando el error humano.
- **Garantizar la equidad en las inversiones:** Implementar un algoritmo que distribuya los dividendos de inversiones conjuntas de forma proporcional al capital aportado por cada miembro.
- **Demostrar la viabilidad tecnológica:** Construir un prototipo funcional que combine dos potentes herramientas:
 - **Solidity:** Actuará como nuestra “ley digital” o un notario automático. Usaremos este lenguaje para crear los *Smart Contracts*, que aseguran que las reglas del grupo se cumplan sin excepciones (por ejemplo, que nadie pueda retirar fondos sin el consenso de los demás).
 - **Python:** Será el “motor de comunicación” del proyecto. Su función es recibir las órdenes sencillas que tú das en la aplicación y traducirlas automáticamente en instrucciones seguras para la blockchain (esto significa que Python hace el trabajo difícil de conectarse con la red y ejecutar los pagos por ti, para que tu experiencia como usuario sea tan fácil como enviar un mensaje por chat).

Nuestra Propuesta: Del Caos al Contrato Inteligente

Para que la historia de Alejandra, Sebastián y Julián tenga un final feliz, nuestra propuesta no se limita a una simple calculadora de gastos. Planteamos un ecosistema donde la tecnología actúa como el garante imparcial de la confianza.

Arquitectura del Sistema: Los Tres Pilares

Nuestra plataforma se sostiene sobre tres componentes que trabajan en total armonía, asegurando que cada peso aportado esté siempre bajo control:

1. **La Bóveda Digital (Solidity):** Utilizaremos contratos inteligentes para crear una “bóveda” inviolable. A diferencia de entregarle el dinero a un amigo, los fondos permanecen en la blockchain y solo pueden liberarse si se cumplen las condiciones preestablecidas por el grupo.
2. **El Motor Analítico (Python):** Queremos que el dinero trabaje para el grupo. Python será el encargado de monitorear las mejores oportunidades de inversión en protocolos DeFi y calcular exactamente cuánto le corresponde a cada miembro, incluyendo los intereses generados, eliminando cualquier duda en la repartición final.
3. **La Ventana al Usuario (Interfaz):** Una aplicación intuitiva donde, en lugar de revisar hilos de WhatsApp, los amigos pueden ver en tiempo real cómo crece su fondo y validar que todos han cumplido con su parte.

Diagrama de Flujo: Un Sistema, Dos Caminos

Nuestra plataforma está diseñada para ser flexible. Dependiendo de la meta del grupo, el sistema puede actuar como un procesador de pagos inmediato o como un fondo de inversión inteligente:

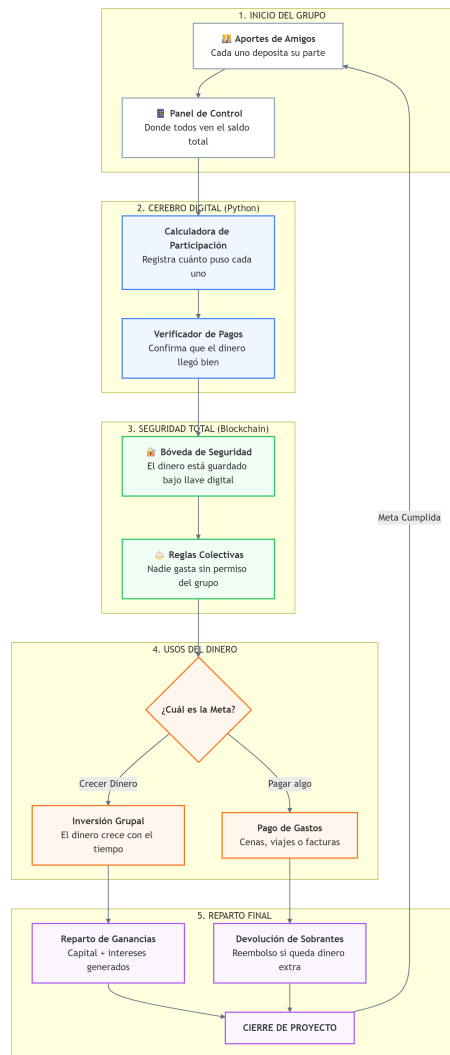


Figura 1: Visualización detallada del flujo operativo del proyecto - Elaborado con Mermaid.js.

Fases de Desarrollo: El Camino a Seguir

Para asegurar la viabilidad de este prototipo, hemos trazado una ruta clara dividida en tres etapas críticas:

- **Etapa 1: Codificación de la Confianza:** Nos enfocaremos exclusivamente en Solidity para programar las reglas de la *Vaca Pool*. El objetivo es tener un contrato que reciba fondos, permita ejecutar pagos a terceros y habilite el envío a protocolos DeFi.
- **Etapa 2: Inteligencia y Conectividad:** Implementaremos el motor en Python. Este decidirá el flujo del dinero basándose en la configuración del grupo y proyectará los repartos finales tras descontar los gastos realizados.
- **Etapa 3: Pulido y Validación:** Realizaremos pruebas de estrés simulando ambos escenarios: un pago rápido de una cena y un ahorro grupal de un mes para un viaje, asegurando que en ambos casos la transparencia sea total.

Fase de Implementación: Interfaz y Simulación del Smart Contract

Durante la fase de desarrollo del prototipo de la aplicación, nos enfocamos en construir una Interfaz de Usuario (UI) y Experiencia de Usuario (UX) que cierre la brecha entre la complejidad de la tecnología Blockchain y la simplicidad que exige el usuario final.

1. Rediseño Premium UI/UX: Estética “Frost & Violet”

Para cumplir con nuestro objetivo de hacer que el dinero fluya de manera intuitiva y amigable, implementamos un diseño inspirado en líderes fintech como Nubank y Nequi, integrando elementos visuales de seguridad propios de MetaMask.

- **Accesibilidad y Claridad:** Se implementó una paleta de colores clara (fondos hielo y superficies blancas) con acentos violetas vibrantes, asegurando que la lectura de saldos y actividades sea inmediata.
- **Glassmorphism:** Uso de transparencias y efectos de desenfoque (*backdrop-filter*) en los menús y ventanas modales, aportando una sensación de tecnología de punta y profundidad en la interfaz.
- **Lenguaje Humano (UX Copywriting):** Reemplazamos la jerga técnica compleja (ej. “Smart Contract” por “Contrato Seguro” y “Hash” por “Recibo Blockchain”), haciendo que cualquier persona, independientemente de su nivel de conocimiento cripto, entienda lo que ocurre con su dinero.

2. Onboarding Educativo: Tour Interactivo Dinámico

La confianza nace del entendimiento. Por ello, programamos en JavaScript puro un sistema de **Spotlight Dinámico** (Recorrido Interactivo). Este sistema oscurece la pantalla e ilumina de forma secuencial cada elemento clave de la aplicación (como el saldo disponible, la sección de inversiones DeFi y el historial blockchain). El código calcula automáticamente las coordenadas de los botones en la pantalla para posicionar el foco y realizar *scroll* automático, guiando al usuario paso a paso por las funcionalidades de los fondos grupales.

3. Simulación y Estructura del Smart Contract en el Frontend

En esta etapa, el Frontend maqueta el comportamiento de los contratos inteligentes (*Smart Contracts*) en Solidity. La lógica de autorización y consenso grupal fue modelada en JavaScript para demostrar el funcionamiento de la “Bóveda Digital”.

A nivel de código, un fondo se define mediante la siguiente estructura de datos que emula el estado en la blockchain:

```
funds = {  
  id_fondo: {  
    title: 'Arriendo apartamento',  
    recaudado: '$1,800,000',  
    meta: '$1,800,000',  
    pct: 100, // Porcentaje de completitud
```

```

hash: '0x2e5d...1f44', // Recibo Blockchain simulado
decision: {
  status: 'Fondo completo',
  next: 'Ejecutar pago al arrendador',
  rule: 'Pago permitido porque todos aportaron',
  action: 'Cerrar fondo'
},
participants: [
  { name: 'Sebastián Ramos', amount: '$600,000', paid: true },
  // ... otros participantes
]
}
}

```

Esta estructura refleja exactamente lo que se almacenará en el Smart Contract:

1. **Transparencia Inmutable:** El hash vincula visualmente el fondo de la app con el recibo en la blockchain.
2. **Consenso Automatizado (decision.rule):** El frontend simula la validación donde el dinero solo se libera (o la tarjeta virtual solo se activa) si el `pct` alcanza el 100 % y todos los participantes han pagado (`paid: true`). Esto cumple con nuestro objetivo específico de *garantizar la equidad y seguridad*.

4. Contrato Inteligente Final: La bóveda digital

El contrato `VacaPool.sol` funciona como la bóveda digital del proyecto. Su papel es guardar las reglas principales del fondo grupal: quién creó la vaca, cuál es la meta, cuánto se ha recaudado, cuánto aportó cada dirección y si el dinero ya puede liberarse.

Para mantener el alcance adecuado de una demo universitaria, el contrato se concentra en cuatro momentos:

1. **Crear el fondo:** el usuario abre la vaca con un nombre y una meta.
2. **Aportar:** cada dirección envía valor al contrato y queda registrada.
3. **Completar la meta:** cuando el acumulado llega al objetivo, el contrato emite el evento `FundCompleted`.
4. **Liberar los recursos:** el creador puede ejecutar `releaseFunds` para enviar el dinero al destinatario correspondiente.

La variable `isReleased` evita que el mismo fondo se libere dos veces, y el evento `FundsReleased` deja una huella clara del cierre del proceso:

```

event FundsReleased(uint256 fundId, address recipient, uint256 amount);

```

```

function releaseFunds(uint256 _fundId, address payable _recipient) public {
  require(_fundId > 0 && _fundId <= fundCount, "El fondo no existe");
  require(_recipient != address(0), "Destinatario invalido");
}

```

```

Fund storage f = funds[_fundId];
require(msg.sender == f.creator, "Solo el creador puede liberar");
require(f.isComplete, "La meta aun no se ha cumplido");
require(!f.isReleased, "El fondo ya fue liberado");

uint256 amount = f.currentAmount;
f.isReleased = true;

(bool sent, ) = _recipient.call{value: amount}("");
require(sent, "No se pudo enviar el dinero");

emit FundsReleased(_fundId, _recipient, amount);
}

```

Esto no pretende ser una auditoría profesional ni un contrato listo para manejar dinero real. La intención es pedagógica: mostrar que un contrato inteligente no es solo una caja donde entra plata, sino un conjunto de reglas que también decide bajo qué condiciones esa plata puede salir.

5. Motor Python Final: La capa que traduce la operación

El archivo `motor.py` representa la capa intermedia entre la aplicación y la blockchain. En una implementación real, esta capa usaría `Web3.py` para conectarse a Polygon, firmar transacciones y consultar el estado del contrato. En esta demo académica, el motor simula ese recorrido completo por consola:

1. Simula conexión con Polygon y el contrato `VacaPool`.
2. Crea un fondo llamado “Viaje Cartagena”.
3. Registra tres aportes de \$600.000 COP.
4. Verifica que la meta de \$1.800.000 COP se cumplió.
5. Simula la liberación del dinero hacia un destinatario.
6. Imprime un resumen auditable de aportes y porcentajes.

Para lograrlo, usamos una estructura simple llamada `FondoDemo`. Esta estructura no reemplaza al Smart Contract; simplemente imita su estado para poder explicar el flujo sin depender de una red blockchain durante la presentación:

```

@dataclass
class FondoDemo:
    fondo_id: int
    nombre: str
    meta: int
    creador: str
    recaudado: int = 0
    completo: bool = False
    liberado: bool = False
    aportes: dict = field(default_factory=dict)

```

Con esto, Python funciona como el “traductor” entre una experiencia sencilla para el usuario y las reglas más estrictas del contrato. El usuario no piensa en `msg.value`, `mapping` o `event`; simplemente ve que hizo un aporte, que se generó un recibo y que el fondo quedó listo para usarse cuando todos cumplieron.

6. Resultado Final de la Implementación

La implementación final queda alineada con Finanzas Descentralizadas porque muestra una lógica financiera completa:

crear fondo → recibir aportes → verificar meta → liberar recursos → dejar trazabilidad

Esa cadena resume la promesa de Grupal: que la confianza del grupo no dependa de perseguir comprobantes por WhatsApp, sino de reglas visibles, estados verificables y eventos que cuentan exactamente qué pasó con el dinero.

Esta implementación valida nuestro objetivo principal: demostrar que la gestión grupal puede ser transparente mediante reglas de código, presentadas a través de una aplicación intuitiva y libre de fricciones.